



CS2023 - Programación III

Implementación detallada del Epic 1 - Feature 4

Grafo computacional, gradientes y parámetros

Rubén Rivas Medina

Abril 2026

Contenido

1. Propósito del documento	3
2. Punto de partida: dependencia con los Features 1, 2 y 3	3
3. Objetivo del Feature 4	3
4. Alcance funcional	3
4.1. Lo que debe quedar implementado	3
4.2. Lo que todavía no forma parte del Feature 4	4
5. Archivos sugeridos	4
6. Interfaz pública esperada	5
7. Convenciones de forma	5
7.1. Notación útil de referencia	5
7.2. Formas en el entrenamiento	5
8. Representación del grafo computacional	6
8.1. Qué significa «grafo» en este proyecto	6
8.2. Estructura interna mínima	6
9. API de entrenamiento	6
9.1. FitOptions	6
9.2. History	7
9.3. EvaluationResult	7
10. Gradientes por capa	7
10.1. Dense	7
10.2. Conv2D	7
10.3. MaxPooling2D	8
10.4. Flatten	8
11. Activaciones y pérdida en backward	8
11.1. Relu	8
11.2. Softmax con CategoricalCrossentropy	8
12. Optimizador mínimo	9
12.1. SGD	9
12.2. Adam como extensión sugerida	9
12.3. RMSProp como extensión sugerida	9
12.4. Sugerencia de diseño para múltiples optimizadores	10
12.5. Estado del optimizador	10
13. API sugerida para gradientes y utilidades	10
13.1. last_gradients()	10
13.2. allclose	10
14. Diseño de Sequential para entrenamiento	11
14.1. Flujo mínimo de fit	11
14.2. evaluate	11
14.3. backward en modelo no compilado	11
15. Implementación paso a paso	11
15.1. Paso 1: cerrar el contrato de gradientes	11
15.2. Paso 2: implementar backward de Dense	12
15.3. Paso 3: implementar backward de Flatten	12
15.4. Paso 4: implementar backward de MaxPooling2D	12
15.5. Paso 5: implementar backward de Conv2D	12
15.6. Paso 6: conectar el optimizador	12

15.7. Paso 7: implementar fit, evaluate y last_gradients	12
15.8. Paso 8: escribir pruebas de comportamiento	12
16. Casos de prueba completos del Feature 4	13
16.1. Qué valida esta prueba	15
17. Criterios de aceptación	15
18. Problemas comunes y errores esperados	16
18.1. Error 1: no guardar cachés suficientes en forward	16
18.2. Error 2: mezclar gradiente de activación con gradiente de parámetro	16
18.3. Error 3: olvidar que MaxPooling no tiene parámetros pero sí tiene backward	16
18.4. Error 4: implementar fit sin actualización real	16
18.5. Error 5: no validar la compatibilidad entre y_true y y_pred	16
18.6. Error 6: propagar Softmax y pérdida por separado de forma inconsistente	16
18.7. Error 7: asumir que el grafo debe ser mucho más complejo de lo necesario	16
19. Conclusión	16

1. Propósito del documento

Este documento describe con detalle la implementación esperada del **Feature 4** del **Epic 1** del proyecto final. En esta etapa la red neuronal ya no debe limitarse a inferir: debe representar un flujo de cálculo entrenable, registrar gradientes y actualizar parámetros en memoria.

El documento está redactado como guía de implementación. Por ello combina decisiones de arquitectura, responsabilidades por capa, flujo de entrenamiento, contratos de API, fragmentos de código, pruebas automatizadas y criterios de aceptación.

Este feature cierra el **Epic 1**. Su propósito es consolidar un modelo secuencial pequeño pero completo, con **forward propagation**, cálculo de pérdida, **backward propagation** y actualización de weights y bias mediante un optimizador mínimo.

2. Punto de partida: dependencia con los Features 1, 2 y 3

Antes de iniciar este feature se asume que el proyecto ya cuenta con:

- backend tensorial estable desde el **Feature 1**,
- operaciones numéricas y conv2d desde el **Feature 2**,
- capas Dense, Conv2D, MaxPooling2D, Flatten, activaciones y Sequential::predict desde el **Feature 3**,
- parámetros visibles mediante parameters(),
- pérdida básica y compile,
- cachés de inferencia o una estructura equivalente para recordar valores necesarios en backward.

Si el **Feature 3** se implementó sin separar con claridad:

- datos intermedios de inferencia,
- parámetros entrenables,
- salida por capa,
- forma de entrada y salida,

entonces este feature se vuelve mucho más costoso. La retropropagación castiga con rapidez cualquier API ambigua.

3. Objetivo del Feature 4

El **Feature 4** persigue siete resultados concretos:

1. Representar un flujo de cálculo suficientemente estructurado como para propagar gradientes capa por capa.
2. Implementar backward para Dense, Conv2D, MaxPooling2D y Flatten.
3. Calcular y almacenar gradientes de weights y bias en las capas entrenables.
4. Incorporar un optimizador mínimo, por ejemplo SGD, que actualice parámetros en memoria, y dejar abierta la posibilidad de añadir Adam y RMSProp como extensiones sugeridas.
5. Implementar un flujo funcional de entrenamiento con fit.
6. Exponer el estado de gradientes más reciente mediante last_gradients().
7. Verificar que la pérdida disminuya sobre un problema pequeño y que los parámetros realmente cambien.

4. Alcance funcional

4.1. Lo que debe quedar implementado

- una representación explícita o implícita del grafo secuencial de cómputo,

- `backward()` del modelo ya compilado,
- gradientes para Dense y Conv2D,
- propagación del gradiente a través de MaxPooling2D y Flatten,
- acumulación o registro de gradientes por nombre de parámetro,
- actualización de parámetros con SGD,
- `evaluate` con al menos pérdida,
- `fit` sobre un dataset pequeño,
- `last_gradients()` con los últimos gradientes calculados,
- un helper como `allclose` para comparar tensores en pruebas,
- pruebas automatizadas con escenarios de éxito y de falla.

4.2. Lo que todavía no forma parte del Feature 4

- optimizadores adicionales más allá de SGD, si el equipo no desea implementarlos como parte de la entrega mínima,
- regularización explícita,
- entrenamiento distribuido,
- soporte obligatorio para mini-batches irregulares,
- gradiente numérico para verificación automática,
- grafos dinámicos complejos o arquitecturas no secuenciales,
- mixed precision o aceleración por GPU.

5. Archivos sugeridos

La siguiente organización corresponde a una vista mínima del árbol del proyecto enfocada en lo necesario para desarrollar y evaluar el **Feature 4**:

```
utec_ai/  
+-- include/  
|   +-- utec/  
|       +-- algebra/  
|           |   +-- shape.h  
|           |   +-- tensor_backend.h  
|           |   +-- tensor_ops.h  
|       +-- nn/  
|           +-- nn_ops.h  
|           +-- nn_interfaces.h  
|           +-- nn_dense.h  
|           +-- nn_convolution.h  
|           +-- nn_pooling.h  
|           +-- nn_flatten.h  
|           +-- nn_activation.h  
|           +-- nn_loss.h  
|           +-- nn_optimizer.h  
|           +-- nn_graph.h  
|           +-- neural_network.h  
+-- tests/  
|   +-- epic1/  
|       +-- feature4_computational_graph_test.cpp
```

El nombre `nn_graph.h` no obliga a construir un grafo simbólico sofisticado. También es válido que el equipo represente el flujo como una secuencia de capas con cachés y funciones `backward`, siempre que el comportamiento observable satisfaga el contrato del feature.

6. Interfaz pública esperada

La API pública debe conservar la experiencia de uso del **Feature 3** y añadir los puntos mínimos para entrenamiento:

```
#include "utec/nn/neural_network.h"

using namespace utec::tf;

int main() {
    Sequential model;
    model.add(layers::Input(Shape{6, 6, 1}));
    model.add(layers::Conv2D(2, {3, 3}, Activation::Relu));
    model.add(layers::MaxPooling2D({2, 2}));
    model.add(layers::Flatten());
    model.add(layers::Dense(2, Activation::Softmax));

    model.compile(
        optimizers::SGD(0.05f),
        losses::CategoricalCrossentropy{});

    auto history = model.fit(x, y, FitOptions{.epochs = 30, .batch_size = 2});
    auto eval = model.evaluate(x, y);
    auto grads = model.last_gradients();

    return (!history.loss.empty() && !grads.empty() && eval.loss >= 0.0f) ? 0 : 1;
}
```

Restricciones importantes:

- `backward()` debe existir en el modelo o en las capas.
- `fit()` debe ejecutar entrenamiento real, no solo llamar a `predict`.
- `last_gradients()` debe devolver gradientes del último paso de entrenamiento.
- `MaxPooling2D` debe participar en `backward`, aunque no exponga parámetros entrenables.
- `SGD` es el optimizador mínimo obligatorio; `Adam` y `RMSProp` pueden ofrecerse como alternativas adicionales.

7. Convenciones de forma

7.1. Notación útil de referencia

- NHWC: {batch, height, width, channels}
- NCHW: {batch, channels, height, width}
- HWIO: {kernel_height, kernel_width, input_channels, output_channels}

El documento asume NHWC para activaciones y HWIO para los kernels convolucionales, en continuidad con los **Features 2** y **3**.

7.2. Formas en el entrenamiento

Durante `fit`, el modelo debe manejar dos tipos de tensores:

- `x`: entradas,
- `y`: etiquetas.

Para clasificación multiclase:

- `x` puede ser `Shape{batch, h, w, c}` o `Shape{batch, features}`,

- y debe ser `Shape{batch, classes}`.

Toda validación de compatibilidad entre salida del modelo y y debe ocurrir antes de empezar la actualización de parámetros.

8. Representación del grafo computacional

8.1. Qué significa «grafo» en este proyecto

En el contexto de este curso, el término **grafo computacional** no obliga a implementar un sistema al nivel de **PyTorch** o **TensorFlow**. Lo mínimo exigible es una estructura donde:

- cada capa conoce su entrada y salida recientes,
- el gradiente fluye en orden inverso,
- cada parámetro entrenable recibe su gradiente asociado,
- el optimizador puede recorrer esos parámetros para actualizarlos.

Una red `Sequential` ya define un grafo lineal. Por eso una solución completamente válida para el curso es:

1. ejecutar forward guardando cachés por capa,
2. calcular el gradiente inicial de la pérdida,
3. recorrer las capas en orden inverso ejecutando backward,
4. recolectar gradientes y aplicar el optimizador.

8.2. Estructura interna mínima

Cada capa entrenable debería poder almacenar al menos:

- último input,
- última salida o preactivación,
- gradiente de weights,
- gradiente de bias.

Las capas no entrenables como `Flatten` y `MaxPooling2D` deberían almacenar:

- información suficiente para reconstruir cómo propagar el gradiente hacia la entrada.

En `MaxPooling2D` esto suele significar:

- índices del máximo por ventana, o
- una máscara del mismo tamaño que la entrada.

9. API de entrenamiento

9.1. FitOptions

Una estructura mínima sugerida es:

```
struct FitOptions {  
    int epochs;  
    int batch_size;  
};
```

Validaciones mínimas:

- `epochs > 0`,
- `batch_size > 0`,

- el dataset no puede estar vacío.

9.2. History

La historia de entrenamiento puede representarse así:

```
struct History {  
    std::vector<float> loss;  
};
```

El contrato mínimo exigible es que `history.loss.size()` coincida con la cantidad de épocas ejecutadas.

9.3. EvaluationResult

Una estructura razonable es:

```
struct EvaluationResult {  
    float loss;  
};
```

Si el equipo quiere añadir más métricas, puede hacerlo, pero no debe romper la interfaz mínima de pérdida.

10. Gradientes por capa

10.1. Dense

Para Dense, si:

- `x`: `Shape{batch, in_features}`
- `w`: `Shape{in_features, units}`
- `b`: `Shape{units}`
- $z = xw + b$

entonces en forma conceptual:

- $dW = x^T * dZ$
- $db = \text{suma por filas de } dZ$
- $dX = dZ * w^T$

La capa debe:

- guardar `x` o `z`,
- aplicar la derivada de la activación,
- calcular `dW`, `db` y `dX`,
- registrar `dW` y `db` como últimos gradientes de sus parámetros.

10.2. Conv2D

Conv2D es la parte más costosa del feature. Conceptualmente necesita:

- gradiente respecto a los pesos,
- gradiente respecto al bias,
- gradiente respecto a la activación de entrada.

En un diseño académico pequeño, es válido implementar el cálculo con bucles explícitos si:

- la forma es correcta,
- el gradiente final es consistente,
- el entrenamiento realmente reduce la pérdida.

No se exige la implementación más rápida. Sí se exige una implementación razonada y verificable.

10.3. MaxPooling2D

MaxPooling2D no tiene parámetros, pero su backward es obligatorio.

La regla conceptual es simple:

- el gradiente de cada ventana debe volver únicamente a la posición que ganó el máximo en forward,
- las demás posiciones de la ventana reciben cero.

Por eso la capa debe recordar qué posición ganó en cada ventana del **forward**.

10.4. Flatten

Flatten no tiene parámetros y su backward es directo:

- recibe un gradiente de rango 2,
- lo devuelve con la forma original del input almacenado en la capa.

En esencia, el backward de Flatten es un reshape inverso.

11. Activaciones y pérdida en backward

11.1. Relu

La derivada de Relu debe valer:

- 1 donde la preactivación es positiva,
- 0 en caso contrario.

Una implementación práctica puede guardar:

- la preactivación, o
- la salida activada,

siempre que eso permita reconstruir correctamente la máscara de la derivada.

11.2. Softmax con CategoricalCrossentropy

Para este proyecto es recomendable aprovechar la simplificación clásica:

$$dL/dz = y_pred - y_true$$

cuando:

- la salida final usa Softmax,
- la pérdida es CategoricalCrossentropy.

Esa decisión reduce mucho la complejidad del **backward** final y es completamente adecuada para el alcance del curso.

12. Optimizador mínimo

12.1. SGD

Una implementación mínima de SGD puede asumir:

```
namespace optimizers {  
  
struct SGD {  
    float learning_rate;  
};  
  
} // namespace optimizers
```

La actualización conceptual es:

```
param = param - learning_rate * grad
```

No se exige momento, **weight decay** ni otras variantes.

12.2. Adam como extensión sugerida

Si el equipo quiere extender el feature sin alterar el mínimo evaluable, puede añadir soporte para Adam.

Una interfaz razonable sería:

```
namespace optimizers {  
  
struct Adam {  
    float learning_rate;  
    float beta1 = 0.9f;  
    float beta2 = 0.999f;  
    float epsilon = 1e-8f;  
};  
  
} // namespace optimizers
```

La idea no es volver Adam obligatorio, sino permitir que el diseño de compile y del optimizador no quede cerrado exclusivamente a SGD.

12.3. RMSProp como extensión sugerida

Otra alternativa razonable es RMSProp, especialmente si el equipo quiere explorar un optimizador adaptativo más simple que Adam.

Una interfaz posible sería:

```
namespace optimizers {  
  
struct RMSProp {  
    float learning_rate;  
    float rho = 0.9f;  
    float epsilon = 1e-8f;  
};  
  
}
```

```
} // namespace optimizers
```

La lógica general consiste en mantener, por parámetro, un promedio móvil de los cuadrados del gradiente para escalar la actualización.

12.4. Sugerencia de diseño para múltiples optimizadores

Si el equipo decide soportar más de un optimizador, una opción pragmática para este proyecto es usar `std::variant` en lugar de herencia clásica.

Por ejemplo:

```
namespace optimizers {  
  
using Optimizer = std::variant<SGD, Adam, RMSProp>;  
  
} // namespace optimizers
```

Eso permite:

- mantener una API clara en compile,
- evitar jerarquías virtuales innecesarias,
- soportar varios optimizadores sin temprar todo Sequential.

12.5. Estado del optimizador

El enunciado principal habla de «manejo del estado mínimo del optimizador». Para SGD simple, ese estado puede ser únicamente la tasa de aprendizaje y la marca de que el modelo fue compilado con ese optimizador.

Si además se implementa Adam, entonces sí habrá que conservar estado por parámetro, por ejemplo estimaciones de primer y segundo momento. Si se implementa RMSProp, también habrá que guardar acumuladores por parámetro. Ese trabajo debe considerarse una extensión, no parte del piso obligatorio.

13. API sugerida para gradientes y utilidades

13.1. `last_gradients()`

Una firma razonable es:

```
[[nodiscard]] std::unordered_map<std::string, Tensor<float>> last_gradients() const;
```

La salida debe seguir la misma convención de nombres que `parameters()`:

- `conv2d_0/weights`
- `conv2d_0/bias`
- `dense_0/weights`
- `dense_0/bias`

`MaxPooling2D` y `Flatten` no deben aparecer en este mapa porque no tienen parámetros entrenables.

13.2. `allclose`

Para las pruebas se recomienda un helper como:

```
bool allclose(const Tensor<float>& a, const Tensor<float>& b, float atol);
```

Su responsabilidad es:

- verificar shape igual,
- comparar elemento a elemento con una tolerancia absoluta,
- devolver false si algún valor difiere más de lo permitido.

No es una función de entrenamiento, pero simplifica mucho las pruebas automatizadas del feature.

14. Diseño de Sequential para entrenamiento

14.1. Flujo mínimo de fit

Una implementación razonable de fit por época es:

1. dividir el dataset en mini-batches,
2. ejecutar forward,
3. calcular pérdida,
4. construir el gradiente inicial de salida,
5. ejecutar backward,
6. aplicar el optimizador,
7. registrar la pérdida.

Para el curso no se exige:

- **shuffle** obligatorio,
- callbacks,
- validación separada,
- **early stopping**.

14.2. evaluate

evaluate puede compartir gran parte del **forward** con predict, pero debe además calcular la pérdida respecto a y.

Es importante que:

- no actualice parámetros,
- no consuma el estado del optimizador,
- valide la compatibilidad entre y_pred y y_true.

14.3. backward en modelo no compilado

El documento principal exige que un modelo no compilado falle si se invoca backward. La excepción recomendada es `std::logic_error`.

Eso obliga a que Sequential tenga al menos un estado interno como:

- compilado / no compilado.

15. Implementación paso a paso

15.1. Paso 1: cerrar el contrato de gradientes

Antes de programar el entrenamiento, el equipo debe decidir:

- dónde se almacenan los gradientes,

- cómo se nombran,
- qué capa es responsable de exponerlos.

Si esta decisión se posterga, `last_gradients()` y el optimizador terminan acoplados de forma innecesaria.

15.2. Paso 2: implementar backward de Dense

Dense es la mejor capa para empezar porque:

- ya usa `matmul`,
- su derivación es más directa,
- permite validar rápido que los parámetros cambian.

Si esta capa no funciona, el entrenamiento completo casi nunca baja la pérdida.

15.3. Paso 3: implementar backward de Flatten

Es la transición más sencilla entre gradientes 2D y tensores espaciales. También sirve como prueba de que el backend de reshape es coherente.

15.4. Paso 4: implementar backward de MaxPooling2D

Aquí el punto crítico es la máscara o los índices del máximo. Si esa información no se guardó en **forward**, habrá que rehacer la capa.

El comportamiento correcto es:

- reenviar el gradiente solo al elemento ganador,
- asignar cero al resto.

15.5. Paso 5: implementar backward de Conv2D

Este es el paso más delicado del feature. Conviene dejarlo para después de validar Dense, Flatten y MaxPooling2D, porque así cualquier falla residual puede localizarse mejor.

Una implementación con bucles anidados es aceptable si es correcta y pasa las pruebas.

15.6. Paso 6: conectar el optimizador

Una vez que las capas producen gradientes correctos, el siguiente paso es aplicar SGD a cada parámetro entrenable del modelo.

Si el equipo decide añadir Adam o RMSProp, conviene hacerlo recién después de cerrar un flujo estable con SGD, porque de lo contrario se vuelve más difícil distinguir si un error viene del cálculo de gradientes o del optimizador.

En este punto debe quedar claro:

- qué parámetros se actualizan,
- con qué gradiente,
- en qué momento del ciclo de entrenamiento.

15.7. Paso 7: implementar fit, evaluate y last_gradients

Con el backward operativo, ya se puede cerrar la API pública del entrenamiento y añadir las utilidades observables del modelo.

15.8. Paso 8: escribir pruebas de comportamiento

La validación final no debe limitarse a compilar. Debe mostrar:

- disminución de pérdida,

- existencia de gradientes,
- actualización real de parámetros,
- fallos controlados ante entradas incompatibles o estados inválidos.

16. Casos de prueba completos del Feature 4

El siguiente archivo de pruebas puede usarse como base evaluable del feature:

```
#include <stdexcept>
#include <string>
#include <string_view>

#include "utec/nn/neural_network.h"

using namespace utec::tf;

namespace {

void expect(bool condition, std::string_view message) {
    if (!condition) {
        throw std::runtime_error(std::string(message));
    }
}

template <class Exception, class Fn>
void expect_throw(Fn&& fn, std::string_view message) {
    try {
        fn();
    } catch (const Exception&) {
        return;
    }
    throw std::runtime_error(std::string(message));
}

} // namespace

int main() {
    Sequential model;
    model.add(layers::Input(Shape{6, 6, 1}));
    model.add(layers::Conv2D(2, {3, 3}, Activation::Relu));
    model.add(layers::MaxPooling2D({2, 2}));
    model.add(layers::Flatten());
    model.add(layers::Dense(2, Activation::Softmax));
    model.compile(
        optimizers::SGD(0.05f),
        losses::CategoricalCrossentropy{});

    auto x = Tensor<float>::from_data(
        Shape{4, 6, 6, 1},
        {
            1, 1, 1, 0, 0, 0,
            1, 1, 1, 0, 0, 0,
            1, 1, 1, 0, 0, 0,
            1, 1, 1, 0, 0, 0,
            1, 1, 1, 0, 0, 0,
            1, 1, 1, 0, 0, 0,
        });
}
```

```

0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,

1, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0,

0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1,
0, 0, 0, 1, 1, 1});

auto y = Tensor<float>::from_data(
    Shape{4, 2},
    {1, 0,
     0, 1,
     1, 0,
     0, 1});

auto params_before = model.parameters();
auto before = model.evaluate(x, y).loss;
auto history = model.fit(x, y, FitOptions{.epochs = 30, .batch_size = 2});
auto after = model.evaluate(x, y).loss;
auto params_after = model.parameters();

expect(history.loss.size() == 30, "history incompleto");
expect(after < before, "la perdida no disminuyo");

auto gradients = model.last_gradients();
expect(!gradients.empty(), "no se registraron gradientes");
expect(gradients.contains("conv2d_0/weights"), "faltan gradientes de conv
weights");
expect(gradients.contains("conv2d_0/bias"), "faltan gradientes de conv bias");
expect(gradients.contains("dense_0/weights"), "faltan gradientes de dense
weights");
expect(gradients.contains("dense_0/bias"), "faltan gradientes de dense bias");
expect(!allclose(
    params_before.at("conv2d_0/weights"),
    params_after.at("conv2d_0/weights"),
    1e-6f),
    "los weights de Conv2D no se actualizaron");
expect(!allclose(
    params_before.at("conv2d_0/bias"),
    params_after.at("conv2d_0/bias"),
    1e-6f),
    "los bias de Conv2D no se actualizaron");
expect(!allclose(
    params_before.at("dense_0/weights"),

```

```

        params_after.at("dense_0/weights"),
        1e-6f),
        "los weights no se actualizaron");
expect(!allclose(
    params_before.at("dense_0/bias"),
    params_after.at("dense_0/bias"),
    1e-6f),
    "los bias no se actualizaron");

expect_throw<std::invalid_argument>(
    [&] {
        auto wrong_y = Tensor<float>::ones(Shape{4, 3});
        (void)model.fit(x, wrong_y, FitOptions{.epochs = 1, .batch_size = 2});
    },
    "fit debio fallar por etiquetas incompatibles");

expect_throw<std::logic_error>(
    [] {
        Sequential bad_model;
        bad_model.add(layers::Input(Shape{4}));
        bad_model.add(layers::Dense(2, Activation::Softmax));
        (void)bad_model.backward();
    },
    "backward debio fallar en un modelo no compilado");
}

```

16.1. Qué valida esta prueba

La prueba anterior cubre ambos tipos de escenarios:

- **éxito:** entrenamiento funcional, pérdida decreciente, gradientes registrados y parámetros actualizados,
- **falla:** etiquetas incompatibles y ejecución de backward en un modelo no compilado.

La intención no es únicamente confirmar que existe una rutina llamada `fit`. Lo que se evalúa es que el entrenamiento realmente modifique el estado del modelo de forma coherente.

17. Criterios de aceptación

El **Feature 4** puede considerarse completo si se cumplen todas estas condiciones:

1. El modelo secuencial soporta backward después de compile.
2. Dense, Conv2D, Flatten y MaxPooling2D participan correctamente en la retropropagación.
3. Los gradientes de weights y bias se calculan y pueden inspeccionarse.
4. SGD actualiza efectivamente los parámetros entrenables.
5. `fit` ejecuta entrenamiento real y devuelve un History consistente.
6. `evaluate` calcula al menos la pérdida.
7. `last_gradients()` expone el último estado de gradientes.
8. Sobre un problema pequeño, la pérdida disminuye.
9. Existe al menos una prueba automatizada con escenarios de éxito y de falla.

18. Problemas comunes y errores esperados

18.1. Error 1: no guardar cachés suficientes en forward

Sin el input reciente, la preactivación o la máscara de **pooling**, el backward se vuelve ambiguo o directamente imposible de implementar bien.

18.2. Error 2: mezclar gradiente de activación con gradiente de parámetro

dX , dW y db son objetos distintos. Si el diseño los mezcla en la misma variable o en la misma interfaz sin claridad, los errores aparecen rápido y son difíciles de depurar.

18.3. Error 3: olvidar que MaxPooling no tiene parámetros pero sí tiene backward

No debe aparecer en `last_gradients()` como parámetro entrenable, pero sí debe transformar correctamente el gradiente que viene de la capa siguiente.

18.4. Error 4: implementar fit sin actualización real

Si `fit` solo calcula pérdida y nunca aplica SGD, la prueba de cambio de parámetros fallará, aunque la estructura general del código parezca correcta.

18.5. Error 5: no validar la compatibilidad entre `y_true` y `y_pred`

Una salida `Shape{batch, 2}` no debe aceptar etiquetas `Shape{batch, 3}`. Ese error debe producir `std::invalid_argument`.

18.6. Error 6: propagar Softmax y pérdida por separado de forma inconsistente

Si la derivada de Softmax y la de `CategoricalCrossentropy` se implementan sin la simplificación estándar, es fácil equivocarse con signos, ejes o normalizaciones.

18.7. Error 7: asumir que el grafo debe ser mucho más complejo de lo necesario

Para este curso, una cadena secuencial con cachés y backward en orden inverso es suficiente. Intentar diseñar un motor genérico sobredimensionado puede consumir mucho tiempo y aumentar el riesgo de errores.

19. Conclusión

El **Feature 4** completa el **Epic 1** al convertir la red neuronal básica en un sistema entrenable. A partir de este punto, el proyecto ya no solo define capas y produce inferencia, sino que calcula pérdida, propaga gradientes, actualiza parámetros y deja evidencia observable del proceso de entrenamiento.

Con este feature cerrado, el equipo dispone de una base funcional para pasar al **Epic 2**, donde el foco deja de estar en la mecánica interna del aprendizaje y se traslada al monitoreo, la aplicación práctica, la persistencia y la documentación experimental.